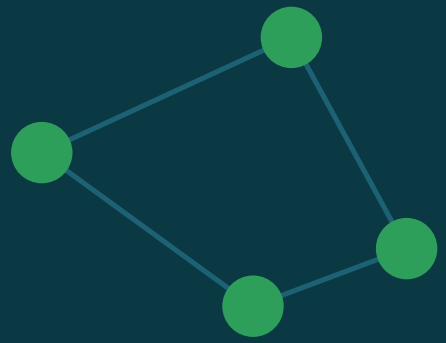
APRESENTAÇÃO · QUESTÕES ENADE

Estruturas de Dados

Seis questões de múltipla escolha — da matéria ao gabarito

SQUAD 9

Bruno da Silva · Cauã Barchi Finotti Franzol · Gustavo Grassini Calabrez · Pedro Henrique Papa Amorim dos Santos



Prof. Fabricio Ricardo Perrella · Bacharelado em Ciência da Computação · USCS

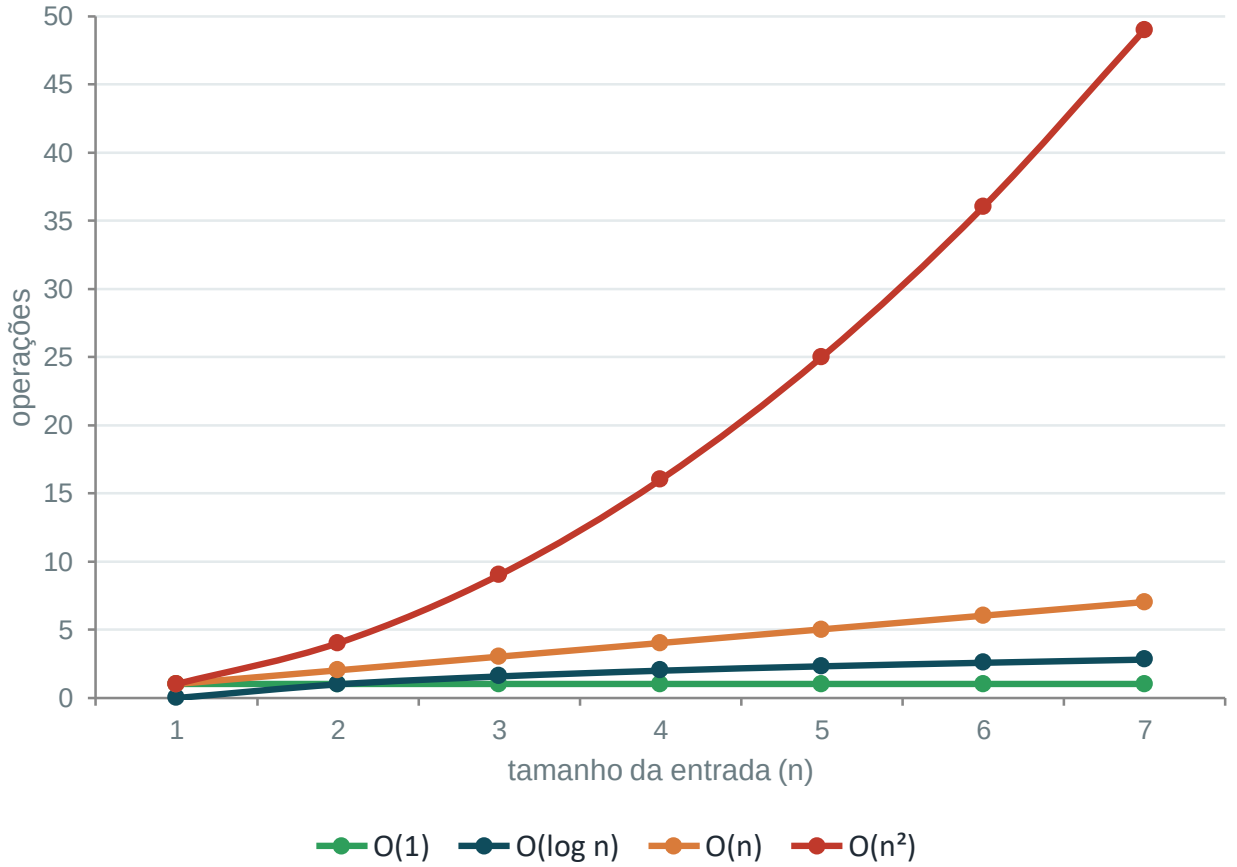
PONTO DE PARTIDA

Como medimos o custo: a notação Big-O

Todas as 6 questões giram em torno de complexidade — este é o mapa para entendê-las

- O(1) — constante: não importa o tamanho.
- O(log n) — cresce devagar (divide pela metade).
- O(n) — linear: proporcional ao tamanho.
- O(n²) — quadrático: explode rápido.

Quanto mais "achatada" a curva, melhor a estrutura escala.



ROTEIRO

As 6 questões

Para cada uma: a matéria, a questão e o gabarito comentado

1. **Listas Encadeadas**
   Asserção-Razão
2. **Pilhas e Filas**
   Ordenação
3. **Árvores (BST / AVL / RN)**
   Afirmativas Múltiplas
4. **Tabela Hash**
   Afirmativa Negativa
5. **Grafos (matriz e lista)**
   Associação
6. **Busca em Grafos (BFS/DFS)**
   Asserção-Razão

1

CONCEITO · QUESTÃO 1

Listas Encadeadas

O que a questão cobra: o custo de remover um nó já conhecido

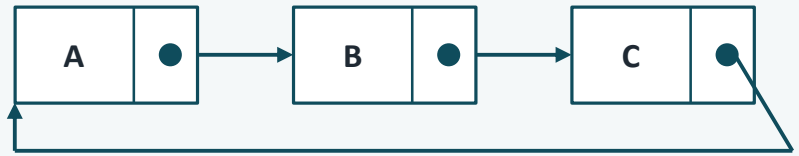
Simplesmente encadeada



Duplamente encadeada



Circular



dupla remove nó conhecido em O(1) (tem ponteiro p/ o anterior)

- Lista simples: cada nó aponta só para o próximo.
- Lista dupla: cada nó aponta para anterior e próximo.
- Remover um nó conhecido: O(1) na dupla, O(n) na simples.
- Sem o ponteiro do anterior, é preciso percorrer a lista.

Analogia: vagões de trem — a lista dupla “enxerga” também o vagão de trás

dupla: O(1)

simples: O(n)

A QUESTÃO · Asserção-Razão · Síntese/Aplicação

1 Listas Encadeadas

SITUAÇÃO-PROBLEMA

Uma equipe está implementando o mecanismo de uma playlist dinâmica na qual o usuário pode remover, a qualquer momento, a faixa atualmente selecionada. A aplicação mantém sempre um ponteiro direto para o nó correspondente à faixa selecionada. A equipe precisa decidir entre implementar a playlist com uma lista simplesmente encadeada ou com uma lista duplamente encadeada, priorizando o menor custo para a operação de remoção dessa faixa.

DADOS DE SUPORTE

```
// Nó de lista simplesmente encadeada
typedef struct No {
   int      dado;
   struct No *prox;       // referência ao próximo nó
} No;

// Nó de lista duplamente encadeada
typedef struct NoD {
   int       dado;
   struct NoD *ant, *prox;  // referências ao anterior e ao próximo
} NoD;
```

COMANDO — Considerando os custos das operações em listas encadeadas, avalie as asserções a seguir e a relação proposta entre elas.

I. Para remover em tempo O(1) a faixa selecionada, da qual já se dispõe do ponteiro para o nó, a lista duplamente encadeada é mais adequada que a simplesmente encadeada.

PORQUE

II. Na lista duplamente encadeada cada nó mantém referências para o nó anterior e para o próximo, permitindo reconectar os vizinhos do nó removido sem percorrer a lista em busca do antecessor.

ALTERNATIVAS

A) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
B) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
C) A asserção I é verdadeira, e a II é falsa.
D) A asserção I é falsa, e a II é verdadeira.
E) As asserções I e II são falsas.

O GABARITO · QUESTÃO 1

1 Listas Encadeadas

Resposta: A — I e II verdadeiras, e a II justifica a I.

JUSTIFICATIVAS

A) (Correta) A asserção I é verdadeira: dispondo-se do ponteiro para o nó, a lista duplamente encadeada remove em O(1), enquanto a simplesmente encadeada precisaria localizar o antecessor percorrendo a lista (O(n)). A asserção II é verdadeira: cada nó da lista dupla guarda referências para anterior e próximo. E a II justifica a I, pois é justamente a referência ao anterior que permite religar os vizinhos e remover em O(1).
B) Incorreta: ainda que ambas sejam verdadeiras, negar o nexo causal é um erro — a estrutura descrita em II é exatamente a razão do desempenho afirmado em I.
C) Incorreta: a asserção II não é falsa; ela descreve corretamente a estrutura de uma lista duplamente encadeada.
D) Incorreta: a asserção I não é falsa; a lista dupla realmente remove um nó conhecido em O(1). Erro de quem supõe que a remoção é sempre O(n), independentemente da estrutura.
E) Incorreta: ambas as asserções são verdadeiras; classificá-las como falsas ignora tanto o custo O(1) da remoção quanto a estrutura de dois ponteiros.

AS ARMADILHAS DOS DISTRATORES

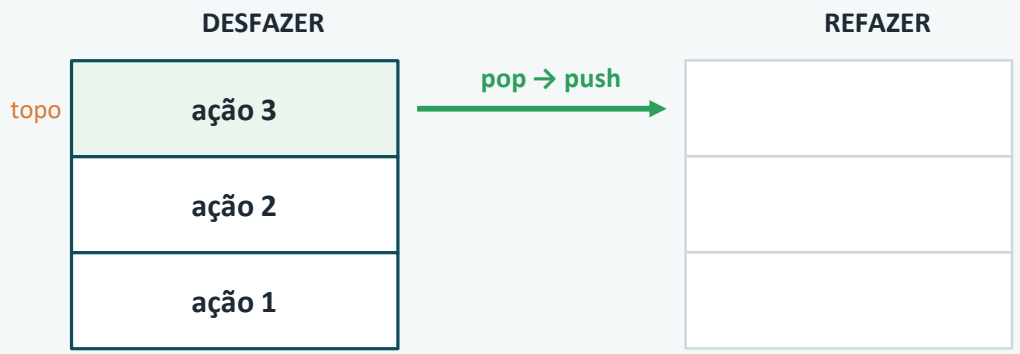
- **B)** ambas verdadeiras, mas nega o nexo causal — só que II é exatamente a razão de I.
- **C)** diz que II é falsa — mas II descreve corretamente a lista dupla.
- **D)** diz que I é falsa — mas a lista dupla realmente remove um nó conhecido em O(1).
- **E)** diz que ambas são falsas — as duas são verdadeiras.

CONCEITO · QUESTÃO 2

2 Pilhas e Filas

O que a questão cobra: a ordem correta das operações com pilha

Duas pilhas (LIFO)



ao desfazer, a ação sai da pilha Desfazer e vai para a Refazer

- Pilha = LIFO: push e pop só no topo, em O(1).
- Desfazer/Refazer usa duas pilhas.
- Antes de remover (pop), checar se a pilha não está vazia.
- A ação desfeita vai para a pilha de Refazer.

Analogia: pilha de pratos — você só mexe no de cima

push / pop = O(1)

2

A QUESTÃO · Ordenação · Avaliação/Análise

Pilhas e Filas

SITUAÇÃO-PROBLEMA

Um editor de texto implementa as funções Desfazer (Undo) e Refazer (Redo) utilizando duas pilhas: a pilha de Desfazer, que armazena as ações já executadas, e a pilha de Refazer. Ao acionar o Desfazer, o sistema precisa reverter a última ação executada e prepará-la para um eventual Refazer.

COMANDO — Considerando o comportamento LIFO das pilhas e o uso das duas estruturas, ordene as etapas que o sistema deve executar, na sequência correta, ao acionar a função Desfazer:

1. Empilhar (push) na pilha de Refazer a ação que foi retirada.
2. Verificar se a pilha de Desfazer não está vazia; se estiver, encerrar (não há o que desfazer).
3. Aplicar ao documento a operação inversa da ação retirada.
4. Retirar (pop) a ação do topo da pilha de Desfazer.

ALTERNATIVAS

A) 4 – 2 – 3 – 1
B) 2 – 4 – 1 – 3
C) 2 – 4 – 3 – 1
D) 4 – 3 – 1 – 2

O GABARITO · QUESTÃO 2

2 Pilhas e Filas

Resposta: C — 2 – 4 – 3 – 1

JUSTIFICATIVAS

A) Incorreta: retira a ação (pop, etapa 4) antes de verificar se a pilha de Desfazer está vazia (etapa 2), arriscando um erro de underflow (remoção em pilha vazia).

B) Incorreta: empilha a ação na pilha de Refazer (etapa 1) antes de aplicar a operação inversa (etapa 3); a ação ficaria disponível para refazer sem ter sido efetivamente desfeita.

C) (Correta) Primeiro verifica se há o que desfazer (2); retira a ação do topo da pilha de Desfazer (4); aplica a operação inversa, efetivando o desfazer (3); e por fim empilha a ação na pilha de Refazer, viabilizando o Refazer posterior (1). Sequência: 2 – 4 – 3 – 1.

D) Incorreta: deixa a verificação de pilha vazia (2) por último, depois de já ter desempilhado e alterado o documento — invertendo a ordem de segurança da operação.

AS ARMADILHAS DOS DISTRATORES

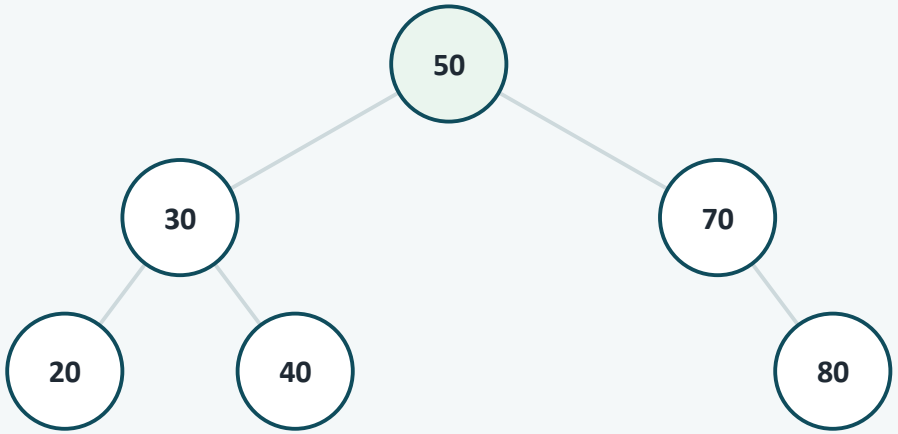
- **A)** 4-2-3-1: faz o pop antes de checar se a pilha está vazia → risco de underflow.
- **B)** 2-4-1-3: empilha no Refazer antes de aplicar o inverso → ação “refazível” sem ter sido desfeita.
- **D)** 4-3-1-2: deixa a checagem de pilha vazia por último, após já alterar o documento.

CONCEITO · QUESTÃO 3

3 Árvores de Busca e Balanceadas

O que a questão cobra: balanceamento e o custo real de cada árvore

Árvore Binária de Busca



esquerda < nó < direita

balanceada → O(log n) · degenerada → O(n)

- BST: esquerda < nó < direita.
- Inserir em ordem degenera a BST → busca O(n).
- AVL e rubro-negra se reequilibram → O(log n) garantido.
- Rubro-negra garante O(log n) — nunca O(1).

Analogia: lista telefônica — você abre no meio e descarta metade a cada passo

balanceada O(log n) **degenerada O(n)**

A QUESTÃO · Afirmativas Múltiplas · Avaliação/Análise

3 Árvores de Busca e Balanceadas

SITUAÇÃO-PROBLEMA

Uma equipe precisa escolher a estrutura para indexar registros que sofrem muitas inserções e remoções, exigindo busca eficiente e com desempenho previsível no pior caso. Foram consideradas a árvore binária de busca (BST) comum e as árvores balanceadas AVL e rubro-negra.

COMANDO — Considerando as propriedades das árvores de busca, analise as afirmativas a seguir:

I. Em uma BST comum, a inserção de chaves em ordem crescente pode degenerar a árvore, elevando o custo da busca a O(n).
II. A árvore AVL mantém, para todo nó, a diferença de altura entre as subárvores esquerda e direita limitada a, no máximo, 1 em módulo, recorrendo a rotações após inserções e remoções.
III. A árvore rubro-negra garante busca em O(1) no pior caso, pois a coloração dos nós elimina a necessidade de percorrer a altura da árvore.
IV. Tanto a AVL quanto a rubro-negra asseguram busca, inserção e remoção em O(log n) no pior caso, diferindo no rigor do balanceamento e, portanto, na frequência de rotações.
É correto apenas o que se afirma em:

ALTERNATIVAS

A) I e II.
B) I, II e IV.
C) III e IV.
D) II e III.

3

O GABARITO · QUESTÃO 3

Árvores de Busca e Balanceadas

Resposta: B — apenas I, II e IV estão corretas.

JUSTIFICATIVA:

A) Incorreta: exclui a afirmativa IV, que é verdadeira.
B) (Correta) Reúne exatamente as três afirmativas verdadeiras: I (a BST degenera com inserção em ordem crescente, levando a busca a O(n)); II (definição do fator de balanceamento da AVL, mantido por rotações); e IV (AVL e rubro-negra garantem O(log n) no pior caso, diferindo no rigor do balanceamento).
C) Incorreta: inclui a afirmativa III (falsa) e exclui as afirmativas I e II (verdadeiras).
D) Incorreta: inclui a afirmativa III (falsa) e exclui as afirmativas I e IV (verdadeiras).

Sobre a III (falsa): a rubro-negra garante O(log n), e não O(1); a coloração limita a altura a cerca de 2·log n, mas a busca ainda percorre essa altura. É o distrator central da questão.

AS ARMADILHAS DOS DISTRATORES

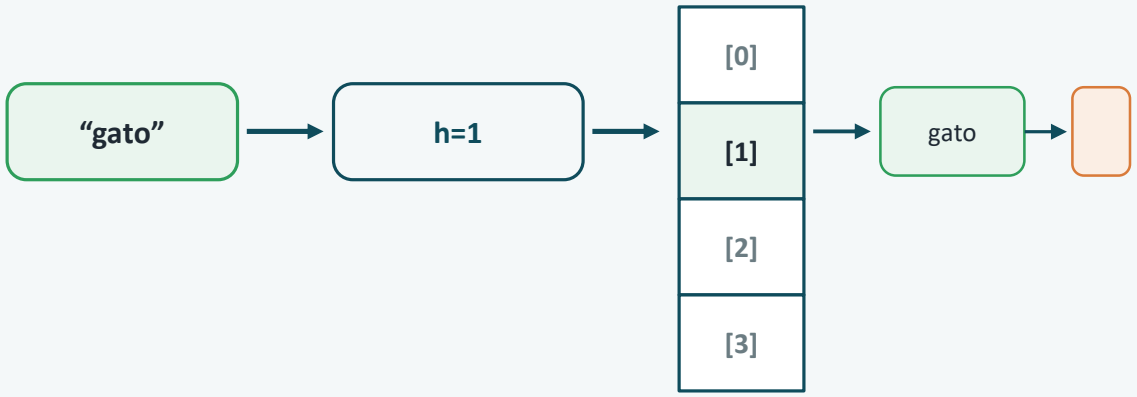
III (falsa): a rubro-negra garante O(log n), não O(1) — a coloração limita a altura a 2·log n, mas a busca ainda percorre essa altura.
A / C / D: erram por excluir alguma verdadeira (IV) ou por incluir a III, que é falsa.

CONCEITO · QUESTÃO 4

4 Tabela Hash e Colisões

O que a questão cobra: distinguir o comportamento real da tabela hash

Chave → função hash → balde



colisão → encadeamento (lista no balde)

- A função hash leva a chave direto a um índice.
- Busca, inserção e remoção: O(1) no caso médio.
- Colisão tratada por encadeamento (lista no balde).
- Não mantém ordem — não faz busca binária.

Analogia: guarda-volumes numerado — a senha leva direto ao armário

busca (médio) O(1)

4

A QUESTÃO · Afirmativa Negativa · Compreensão/Conhecimento

Tabela Hash e Colisões

SITUAÇÃO-PROBLEMA

Um desenvolvedor implementa uma tabela hash para armazenar pares chave-valor em uma aplicação com alto volume de consultas. Ele analisa o papel da função hash, do fator de carga e das técnicas de tratamento de colisões.

COMANDO — Sobre tabelas hash e o tratamento de colisões, assinale a alternativa INCORRETA:

A) No tratamento por encadeamento separado, cada posição da tabela referencia uma lista que reúne todas as chaves que colidiram naquele índice.
B) O fator de carga é a razão entre o número de elementos armazenados e o número de posições da tabela, e o seu aumento tende a elevar o número de colisões.
C) Uma função hash bem projetada distribui as chaves de maneira uniforme pelas posições, reduzindo a probabilidade de colisões.
D) Com uma função hash adequada, a busca em uma tabela hash tem custo O(log n) no caso médio, pois as chaves são mantidas internamente ordenadas para permitir busca binária.

O GABARITO · QUESTÃO 4

4 Tabela Hash e Colisões

Resposta: D — é a afirmação INCORRETA (o que se pede).

JUSTIFICATIVAS

A) Afirmação verdadeira: descreve com precisão o tratamento de colisões por encadeamento separado.
B) Afirmação verdadeira: define corretamente o fator de carga e o seu efeito sobre a taxa de colisões.
C) Afirmação verdadeira: a uniformidade da distribuição é a principal qualidade de uma boa função hash.
D) (INCORRETA — é o gabarito) A busca em tabela hash tem custo O(1) no caso médio, e não O(log n). Além disso, a tabela hash não mantém as chaves ordenadas nem realiza busca binária — esse é o comportamento das árvores de busca balanceadas. A alternativa confunde a tabela hash com uma estrutura ordenada.

AS ARMADILHAS DOS DISTRATORES

A, B, C: são afirmações verdadeiras (encadeamento, fator de carga e uniformidade da função hash) — por isso não são a resposta.

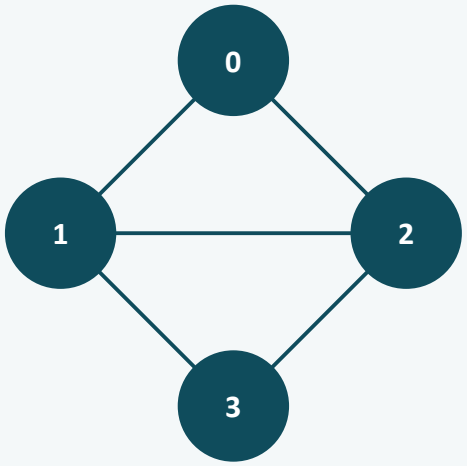
5

CONCEITO · QUESTÃO 5

Grafos: Matriz e Lista

O que a questão cobra: qual representação usar em cada situação

Grafo — duas representações



Matriz

0	1	1	0
1	0	1	1
1	1	0	1
0	1	1	0

matriz $O(V^2)$ · lista $O(V+E)$

- Matriz de adjacência: aresta em O(1), espaço $O(V^2)$.
- Lista de adjacência: espaço O(V+E), boa p/ esparsos.
- Iterar sobre vizinhos: O(grau) na lista.
- Denso → matriz; esparso → lista.

Analogia: rede social — pessoas são vértices e amizades são arestas

matriz $O(V^2)$ | **lista O(V+E)**

5

A QUESTÃO · Associação · Compreensão/Conhecimento

Grafos: Matriz e Lista

SITUAÇÃO-PROBLEMA

Ao modelar diferentes sistemas como grafos, um analista precisa escolher a representação mais adequada — matriz de adjacência ou lista de adjacência — conforme a densidade do grafo e as operações mais frequentes.

COMANDO — Relacione cada necessidade da Coluna 1 à representação de grafos mais adequada, descrita na Coluna 2

Coluna 1 — Necessidade

1. Consultar em tempo O(1) se existe a aresta entre dois vértices dados.
2. Minimizar o uso de memória em um grafo esparso (poucas arestas).
3. Iterar eficientemente sobre os vizinhos de um vértice.

Coluna 2 — Representação (razão)

( ) Lista de adjacência — o espaço cresce com O(V + E), evitando desperdício.
( ) Lista de adjacência — percorre-se apenas a lista do próprio vértice, em O(grau(v)).
( ) Matriz de adjacência — o acesso direto à célula [u][v] responde em O(1).

ALTERNATIVAS (de cima para baixo)

A) 1 – 2 – 3
B) 2 – 3 – 1
C) 3 – 2 – 1
D) 1 – 3 – 2

O GABARITO · QUESTÃO 5

5 Grafos: Matriz e Lista

Resposta: B — 2 – 3 – 1

JUSTIFICATIVAS

Associação correta: a economia de memória em grafo esparso (necessidade 2) corresponde à lista de adjacência com espaço O(V + E); a iteração sobre vizinhos (necessidade 3) corresponde a percorrer a lista do próprio vértice em O(grau); e a consulta O(1) de aresta (necessidade 1) só a matriz de adjacência oferece, pelo acesso direto à célula [u][v]. Lendo a Coluna 2 de cima para baixo, obtém-se 2 – 3 – 1.

A) Incorreta: associa a consulta de aresta (matriz) à economia de memória, o que é falso — a matriz consome $O(V^2)$.
B) (Correta) Sequência 2 – 3 – 1, conforme a análise acima.
C) Incorreta: troca entre si as duas vantagens da lista e desloca a matriz de posição.
D) Incorreta: parte de uma associação errada na primeira lacuna (coloca a necessidade 1 na lista de economia de memória), desalinhando toda a sequência.

AS ARMADILHAS DOS DISTRATORES

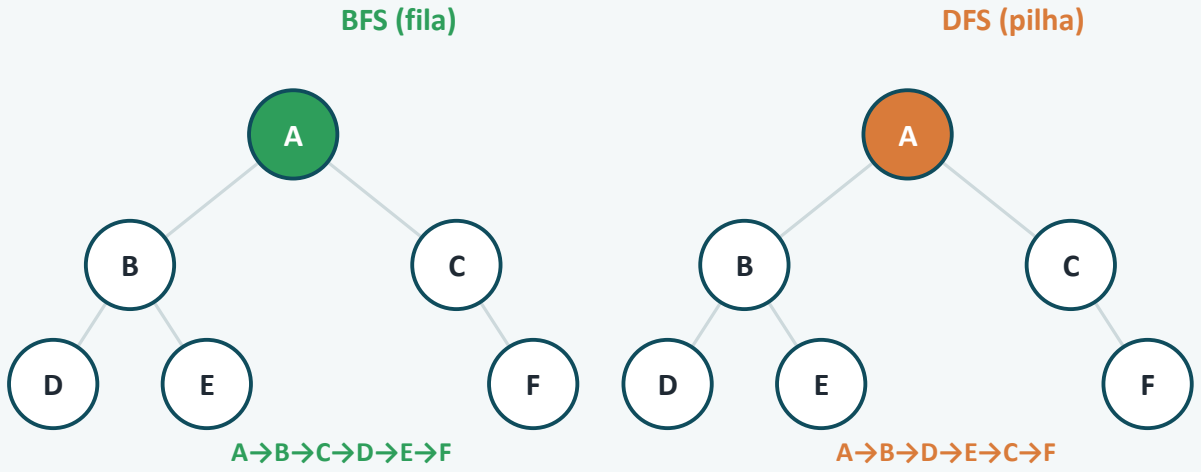
A / D: colocam a necessidade 1 (consulta de aresta) na lista de economia de memória — mas a matriz é que consome $O(V^2)$.
C: troca entre si as duas vantagens da lista, desalinhando a sequência.

6

CONCEITO · QUESTÃO 6

Busca em Grafos: BFS e DFS

O que a questão cobra: qual travessia resolve o menor caminho



menor caminho (não ponderado) = BFS

- BFS usa fila e explora o grafo por níveis.
- DFS usa pilha/recursão e vai o mais fundo primeiro.
- Menor caminho (grafo não ponderado) = BFS, não DFS.
- Ambas percorrem o grafo em O(V+E).

Analogia: BFS = onda que se espalha; DFS = explorar um labirinto até o fim

BFS / DFS = O(V+E)

6

A QUESTÃO · Asserção-Razão · Síntese/Aplicação

Busca em Grafos: BFS e DFS

SITUAÇÃO-PROBLEMA

Um desenvolvedor júnior sugeriu implementar o recurso “grau de conexão” de uma rede social — que informa o menor número de intermediários entre dois usuários, em um grafo não ponderado (cada amizade tem o mesmo peso) — utilizando a busca em profundidade (DFS). A equipe precisa avaliar criticamente essa proposta.

COMANDO — Considerando os algoritmos de busca em grafos, avalie as asserções a seguir e a relação proposta entre elas

I. Para encontrar o menor caminho em número de arestas entre dois usuários em um grafo não ponderado, a busca em profundidade (DFS) é a escolha adequada.

PORQUE

II. A DFS percorre o grafo aprofundando-se ao máximo em um ramo antes de retroceder (backtracking) para explorar caminhos alternativos.

ALTERNATIVAS

A) As asserções I e II são verdadeiras, e a II é uma justificativa correta da I.
B) As asserções I e II são verdadeiras, mas a II não é uma justificativa correta da I.
C) A asserção I é verdadeira, e a II é falsa.
D) A asserção I é falsa, e a II é verdadeira.
E) As asserções I e II são falsas.

O GABARITO · QUESTÃO 6

6 Busca em Grafos: BFS e DFS

Resposta: D — a asserção I é falsa e a II é verdadeira.

JUSTIFICATIVAS

A) Incorreta: a asserção I é falsa, o que já elimina esta alternativa.
B) Incorreta: pelo mesmo motivo — a asserção I é falsa.
C) Incorreta: a asserção II é verdadeira (descreve corretamente o funcionamento da DFS), e não falsa.
D) (Correta) A asserção I é falsa: a DFS não garante o menor caminho em número de arestas; quem resolve esse problema em grafos não ponderados é a busca em largura (BFS), que explora o grafo por níveis (distâncias crescentes a partir da origem). A asserção II é verdadeira: descreve corretamente a DFS. Como a I é falsa e a II é verdadeira, a alternativa correta é a D.
E) Incorreta: a asserção II é verdadeira, logo não é o caso de ambas serem falsas.

AS ARMADILHAS DOS DISTRATORES

A / B: assumem a asserção I como verdadeira — mas a DFS não serve para o menor caminho.
C: diz que II é falsa — mas II descreve corretamente a DFS.
E: diz que ambas são falsas — a II é verdadeira.

PARA FECHAR

Toda escolha de estrutura é uma escolha de custo.

Entender O(1), O(log n) e O(n) é o que permite ler qualquer questão de Estruturas de Dados — e é o fio que conecta as seis questões que apresentamos.

Obrigado!

Squad 9 · Bruno · Cauã · Gustavo · Pedro Henrique